

Báo cáo rà soát test code

<!-- framework: pytest | run_id: run_20260618_183830_559941 -->

Framework: pytest | Run ID: run_20260618_183830_559941

Tóm tắt kỹ thuật

- Framework: pytest
- Kết quả chạy: 20/20 passed, coverage 100 % (không có lỗi execution).
- Kiểm tra chính: Đánh giá các test so với yêu cầu, test plan, và source code.
- Lỗi chặn lớn nhất: Không có lỗi chặn; tất cả test đều chạy được và đáp ứng các kịch bản đã định.

Lỗi nghiêm trọng

Không phát hiện.

Test còn thiếu

- factorial: chưa có test kiểm tra hành vi với số nguyên rất lớn (ví dụ $n=10000$) để đánh giá hiệu năng hoặc khả năng overflow, mặc dù có test TC-018 cho $n=1000$ nhưng chỉ kiểm tra >0 và kiểu int.
- clamp: chưa có test cho trường hợp minimum và maximum là các kiểu không thể so sánh (ví dụ `minimum='a'`, `maximum='z'`) để xác nhận thông báo lỗi cụ thể.
- safe_divide: chưa có test cho trường hợp numerator hoặc denominator là `float('nan')` hoặc `float('inf')` để kiểm tra xử lý giá trị đặc biệt.

Assertion yếu

- TC-018 (factorial với số lớn):
- Bằng chứng: `assert result > 0` và `assert isinstance(result, int)` chỉ kiểm tra tính dương và kiểu, không xác nhận giá trị chính xác của `factorial(1000)`.
- Ảnh hưởng: Có thể bỏ qua lỗi tính toán sai mà vẫn vượt qua test.
- TC-019 (clamp với kiểu không phải số):
- Bằng chứng: `with pytest.raises(TypeError): clamp('a', 0, 10)` chỉ kiểm tra có ném `TypeError` mà không kiểm chứng thông điệp lỗi, trong khi tài liệu yêu cầu thông báo cụ thể.
- Ảnh hưởng: Nếu hàm thay đổi để ném `ValueError` hoặc thông báo khác, test vẫn sẽ pass mà không phát hiện sai lệch yêu cầu.

Rủi ro maintainability

- Phụ thuộc vào kiểu dữ liệu mặc định của Python: Các test như `clamp('a', 0, 10)` dựa vào việc Python ném `TypeError` khi so sánh str và int. Nếu trong tương lai hàm được cập nhật để chuyển đổi kiểu hoặc xử lý chuỗi, test sẽ thất bại mà không có mô tả rõ ràng.
- Thiếu kiểm tra thông điệp lỗi chi tiết: Một số test chỉ kiểm tra loại ngoại lệ mà không kiểm tra match (ví dụ TC-019, TC-020). Điều này làm giảm khả năng phát hiện thay đổi hành vi không mong muốn.

Gợi ý sửa ngay

1. Cải thiện TC-018:

```
def test_factorial_TC_018():
```

```
    result = factorial(1000)
```

```
    # Kiểm tra giá trị chính xác (có thể dùng math.factorial để so sánh)
```

```
    import math
```

```
    assert result == math.factorial(1000)
```

2. Thêm kiểm tra thông điệp lỗi cho TC-019 và TC-020:

```
with pytest.raises(TypeError, match="unsupported operand type"):
```

```
    clamp('a', 0, 10)
```

```
with pytest.raises(TypeError, match="unsupported operand type"):
```

```
    safe_divide('10', '2')
```

3. Thêm test cho factorial với số nguyên cực lớn (hiệu năng/overflow):

```
def test_factorial_large_performance():
```

```
    import time
```

```
    start = time.time()
```

```
    factorial(5000)
```

```
    assert time.time() - start < 1.0 # ví dụ giới hạn thời gian
```

4. Thêm test cho clamp khi minimum và maximum là chuỗi không thể so sánh:

```
def test_clamp_invalid_bounds_type():
```

```
    with pytest.raises(TypeError):
```

```
        clamp(5, 'a', 'z')
```

5. Thêm test cho safe_divide với nan và inf:

```
import math
```

```
def test_safe_divide_nan_inf():
```

```
    assert math.isnan(safe_divide(math.nan, 1))
```

```
    assert safe_divide(1, math.inf) == 0.0
```

Các đề xuất trên giúp tăng độ bao phủ, độ chắc chắn của các assertion và giảm rủi ro khi mã nguồn thay đổi trong tương lai.